

## RI: Small: Novel algorithms for practical deep reinforcement learning

PI: Jian Peng; co-PI: Alexander Schwing

**Project Summary:** Reinforcement learning studies algorithms which learn policies that encode decisions for complex environments. With the recent advances in deep learning, it is natural to incorporate deep neural networks into reinforcement learning algorithms and assist perception of the environment and decision making. Deep Q-learning, possibly a first attempt of this combination, has been proved to be effective on a variety of problems which traditional artificial intelligence approaches were unable to solve. Notable examples include human-level performance game playing, AlphaGo and robotic manipulation. However, learning policies for these tasks requires a huge amount of computational resources, which makes deep reinforcement learning algorithms impractical for many other real-world tasks.

In this proposal, we will focus on three new algorithmic approaches to accelerate deep reinforcement learning, in particular deep Q-learning. First, we will develop optimality tightening, a constrained optimization technique which encourages fast reward propagation in deep Q-learning. Second, we will introduce new and flexible neural network structures for multitask deep reinforcement learning and transfer learning by sharing key network components across tasks. Third, we will develop algorithms to incorporate human experience or guidance into the deep Q-learning such that agents can mimic high-reward human behavior in complex tasks. We will validate our work on standard reinforcement learning benchmarks, including challenging Atari games, the Mujoco physics engine, driving simulators, *etc.*, and compare to existing approaches. The methods proposed in each of our three aims are complementary to each other and are also mostly orthogonal to recent algorithmic advances in this field. These new approaches will enable us to train policies for real-world, complex tasks with only small scales of computational resources and will result in important and new algorithms that will vastly improve state-of-the-art deep reinforcement learning and broaden the applicability to many real-world problems in science and engineering.

**Intellectual Merit:** Deep reinforcement learning has recently been proposed to solve complex decision-making tasks in dynamic environments. One draw back of such methods is the demand of large-scale computing resources, thus limiting its application to problems where trajectory data are cheap to obtain. Therefore, we see the pressing need for data efficiency of deep reinforcement learning algorithms. This research leverages prior knowledge from past experience, similar tasks and human demonstrations to establish new algorithms to improve sample complexity of deep Q-learning. These methods are also synergistic to many existing approaches developed in this field.

**Broader Impact:** Development of intelligent systems for improving human life quality is a principal objective in artificial intelligence. Practical algorithms of deep reinforcement learning will broaden the applicability of such technology to real-world problems, such as health care, robotics, autonomous driving and efficient energy control. Furthermore, the research proposed by the PIs will allow them to attract undergraduate and graduate students and support the students academic success. In addition, high school outreach promotes enrollment of female students in science and engineering programs.

# 1 Introduction

Artificial Intelligence (AI) is of great interest to many industries from entertainment and electronic services, computer gaming, chatbot-assisted systems to autonomous driving, and many more. The recent advances of supervised deep learning techniques [1] in computer vision, speech recognition and natural language processing have tremendously improved the performance on challenging tasks, including image processing [2], speech-based translation [3] and language modeling [4]. The core idea of deep learning is to use artificial neural networks to model complex hierarchical or compositional data abstractions and representations from raw input data [5]. Together with the ever-increasing data repositories and the available computational power, improvements of deep learning have led to a large number of successful electronic products, including PlayStation Move, Kinect and Mobileye driver assistance systems. However, we are still far from building intelligent solutions for many real-world challenges, such as autonomous driving, human-computer interaction and automated decision making, in which software agents need to consider interactions with a dynamic environment and take actions towards goals. Reinforcement learning [6, 7, 8, 9] studies these problems and algorithms to learn policies to make decisions so as to maximize a reward signal from the environment. One of the promising reinforcement learning algorithms is Q-learning [10, 11].

Combining deep learning and reinforcement learning is a natural fit: deep neural networks extract important feature mappings from the environment, and reinforcement learning algorithms make decisions based on the features. Deep Q-learning [12, 13, 14], possibly a first attempt of this combination, has been proved to be effective on a few problems which traditional AI approaches were unable to solve. Notable examples include human-level performance game playing, AlphaGo [15] which defeated the human world champion Lee Sedol in 2016, and the ongoing DeepMind health-care project. Despite their successes, these applications require a huge amount of computational resources. For example, even for a simple Atari game, the deep Q-learning algorithm needs to play tens to hundreds of millions of episodes to achieve a reasonable performance [16]. In AlphaGo, researchers from Google DeepMind trained the model based on a database of game records of advanced players and more than 30 millions of self-played games [15]. The sheer enormity of the computational complexity of current deep reinforcement learning algorithms is a major bottleneck for its application to other real-world tasks. Another limitation of current approaches is that they cannot transfer learned policies from a well-trained task to a relevant new task, while the human brain can easily exploit previously acquired knowledge for new problems. Development of new, efficient deep reinforcement learning algorithms is thus a pressing need.

Here we propose to design and develop new algorithmic approaches to accelerate deep reinforcement learning, in particular deep Q-learning. These new approaches will enable us to train policies for real-world, complex tasks with only small scales of computational resources. Our three specific aims are:

**1. To develop new deep Q-learning algorithms with fast reward propagation** (Sec. 2.1). While current deep Q-learning algorithms rely on a set of experienced replays, they only consider a single forward step for the Bellman optimality error minimization. Multi-step deep Q-learning and eligibility trace algorithms still require further careful tuning to work on complex tasks. To better exploit long-term high-reward strategies from past experience, we will design ‘optimality tightening,’ a new technique to capture multiple forward steps of the replays to speed up the learning process. We will explore several possible solutions and provide preliminary results on challenging Atari games. This technique is general and will be easily extended for specific tasks.

**2. To develop new deep neural network architectures for more efficient information sharing** (Sec. 2.2). Akin to the first aim, experience on a task should provide insights for similar tasks. In order to exploit such prior information, we require a mechanism to share information across different tasks. We will introduce new neural network structures for multitask deep reinforcement learning and transfer learning by sharing key network components across tasks. For similar tasks, we will design intermediate neural network layers that take visual representations to learn decision strategies shared across tasks, connected with tasks-specific

environment mapping and decision making networks.

**3. To develop new approaches for learning from human guidance** (Sec. 2.3). Human guidance provides valuable prior information or successful examples for complex tasks. Normally, only a small set of demonstration examples are provided since the accessibility of human guidance is often expensive and time-consuming to come by. To exploit human guidance, we propose to develop efficient algorithms to extract information from sparse human guidance and utilize them to regularize Q-learning such that agents can mimic high-reward human behaviors. The proposed methods will not only enable faster training of Q-networks, but also provide further insights for human-robot interactions.

## 1.1 Background and Significance

Reinforcement learning considers agents which are able to take a sequence of actions in an environment. By taking actions and experiencing at most one scalar reward per action, their task is to learn a policy which allows them to act such that a high cumulative reward is obtained over time.

More precisely, consider an agent operating over time  $t \in \{1, \dots, T\}$ . At time  $t$  the agent is in an environment state  $s_t$  and reacts upon it by choosing action  $a_t \in \mathcal{A}$ . The agent will then observe a new state  $s_{t+1}$  and receive a numerical reward  $r_t \in \mathcal{R}$ . Throughout, we assume the set of possible actions, *i.e.*, the set  $\mathcal{A}$ , to be discrete.

A well established technique to address the aforementioned reinforcement learning task is Q-learning [10, 11]. Generally, Q-learning algorithms maintain an action-value function, often also referred to as Q-function,  $Q(s, a)$ . Given a state  $s$ , the action-value function provides a ‘value’ for each action  $a \in \mathcal{A}$  which estimates the expected future reward if action  $a \in \mathcal{A}$  is taken. The estimated future reward is computed based on the current state  $s$  or a series of past states  $s_t$  if available.

The core idea of Q-learning is the use of the Bellman equation as a characterization of the optimal future reward function  $Q^*$  via a state-action-value function

$$Q^*(s_t, a) = \mathbb{E}[r_t + \gamma \max_{a'} Q^*(s_{t+1}, a')]. \quad (1)$$

Hereby the expectation is taken w.r.t. the distribution of state  $s_{t+1}$  and reward  $r_t$  obtained after taking action  $a$ , and  $\gamma$  is a discount factor. Intuitively, reward for taking action  $a$  plus best future reward should equal the best total return from the current state.

The choice of Q-function is crucial for the success of Q-learning algorithms. While classical methods use linear Q-functions based on a set of hand-crafted features of the state, more recent approaches use nonlinear deep neural networks to automatically mine intermediate features from the state [17, 18, 13, 14]. This change has been shown to be very effective for many applications of reinforcement learning. However, automatic mining of intermediate representations comes at a price: larger quantities of data and more computational resources are required. Even though it is sometimes straightforward to extract large amounts of data, *e.g.*, when training on video games, for successful optimization, it is crucial that the algorithms operate on un-correlated samples from a dataset  $\mathcal{D}$  for stability. A technique called “experience replay” [19, 20] encourages this property and quickly emerged as a standard step in the well-known deep Q-learning framework [13, 14]. Experience replays are stored as a dataset  $\mathcal{D} = \{(s_j, a_j, r_j, s_{j+1})\}$  which contains state-action-reward-future state-tuples  $(s_j, a_j, r_j, s_{j+1})$ , including past observations from previous plays. Note that we use the subscript  $j$  to index a tuple in the replay memory, while subscript  $t$  refers to the current timepoint the agent is at.

The characterization of optimality given in Eq. (1) combined with an “experience replay” dataset  $\mathcal{D}$  results in the following iterative algorithmic procedure [13, 14]: start an episode in the initial state  $s_0$ ; sample a mini-batch of tuples  $\mathcal{B} = \{(s_j, a_j, r_j, s_{j+1})\} \subseteq \mathcal{D}$ ; compute and fix the targets  $y_j = r_j + \gamma \max_a Q_{\theta-}(s_{j+1}, a)$  for each tuple using a recent estimate  $Q_{\theta-}$  (the maximization is only considered if

$s_j$  is not a terminal state); update the Q-function by optimizing the following program w.r.t. the parameters  $\theta$  typically via stochastic gradient descent:

$$\min_{\theta} \sum_{(s_j, a_j, r_j, s_{j+1}) \in \mathcal{B}} (Q_{\theta}(s_j, a_j) - y_j)^2. \quad (2)$$

After having updated the parameters of the Q-function we perform an action simulation either choosing an action at random with a small probability  $\epsilon$ , or by following the strategy  $\arg \max_a Q_{\theta}(s_t, a)$  which is currently estimated. This strategy is also called the  $\epsilon$ -greedy policy. We then obtain the actual reward  $r_t$ . Subsequently we augment the replay memory with the new tuple  $(s_t, a_t, r_t, s_{t+1})$  and continue the simulation until this episode terminates or reaches an upper limit of steps, and we restart a new episode. When optimizing w.r.t. the parameter  $\theta$ , a recent Q-network is used to compute the target  $y_j = r_j + \gamma \max_a Q_{\theta-}(s_{j+1}, a)$ . This technique is referred to as ‘semi-gradient descent,’ *i.e.*, the dependence of the target on the parameter  $\theta$  is ignored.

**Development of new deep Q-learning algorithms with fast reward propagation.** Investigating the cost function given in Eq. (2) more carefully, we observe that it operates on a set of short one-step sequences, each characterized by the tuple  $(s_j, a_j, r_j, s_{j+1})$ . Intuitively, each step encourages an update of the parameters  $\theta$ , such that the action-value function for the chosen action  $a_j$ , *i.e.*,  $Q_{\theta}(s_j, a_j)$  is closer to the obtained reward plus the best achievable future reward, *i.e.*,  $y_j = r_j + \gamma \max_a Q(s_{j+1}, a)$ . As we expect from the Bellman equation, it is instructive to interpret this algorithm as propagating reward information from time  $j + 1$  backwards to time  $j$ . To understand the shortcomings of this procedure consider a situation where the agent only receives a reward once reaching a target in a maze. Further let  $|P|$  characterize the shortest path from the agents initial position to the target. For a long time, no real reward is available and the aforementioned algorithm propagates randomly initialized future rewards. Once the target is reached, real reward information is available. Due to the cost function and its property of propagating reward time-step by time-step, it is immediately apparent that it takes at least an additional  $\mathcal{O}(|P|)$  iterations until the observed reward impacts the initial state. Sharper characterizations for the speed of propagation are possible.

In Sec. 2.1 we propose a technique which allows fast reward propagation from past and future state-action pairs. We achieve this improvement by taking advantage of longer state-action-reward-sequences which are readily available in the ‘experienced replay memory’ and build optimality upper and lower bounds to restrain Q-learning. Even though we expect to see substantial improvements on sequences where rewards are sparse or only available at terminal states, we demonstrate significant speedups also for situations where rewards are obtained frequently. This is intuitive as the Q-function represents an estimate for any reward encountered in the future. Based on the proposed optimality tightening framework, we further introduce potential variants that can broaden the applicability to other problems.

**Design of new deep neural network architectures for more efficient information sharing.** Moreover, we also want to train faster on new tasks that are similar to previously considered environments or that possess similar decision rules. Our intuition is based on the fact that we all perform well on tasks that require skills related to situations that we have mastered before. To achieve faster training on previously unseen tasks we need to leverage information from representations that yield successful results in other environments. To achieve this target we propose Q-functions for joint training across multiple tasks in Sec. 2.2. Those Q-functions are composed out of three parts, a visual representation extraction, a decision logic, and a decision execution. Every task has a separate representation extraction and decision execution, but the decision logic is shared. Training on a set of tasks follows the standard procedure and the parameters of all three parts are updated. Learning for a previously unseen environment requires adjusting the visual representation and the decision execution. We will explore training techniques such as [21] in our setting. With the proposed new deep neural network architectures, we will be able to transfer knowledge from previously learned tasks to new and unobserved tasks, thus greatly reducing the training time.

**Development of efficient algorithms to exploit human guidance for deep Q-learning.** Until today, most algorithmic agents can only perform well on simple tasks with short-term goals. In contrast, we can easily solve many complex tasks that reinforcement learning and control algorithms fail to master. It is thus crucial to develop methods which incorporate human expertise on such tasks to assist reinforcement learning. However, it is usually difficult and expensive to obtain a large set of human demonstrations. In Sec. 2.3 we present algorithmic approaches to extract information from sparse human guidance and assist deep Q-learning. By modeling human guidance as a regularization and/or via a reward-shaping potential, our approaches are flexible for different types of human guidance. In addition, we also hope to explore the effectiveness of potential types of human-agent interactions to further improve the performance of the human-guided deep Q-learning pipeline.

**Overall,** the combination of all three concepts proposed in Sec. 2.1, Sec. 2.2 and Sec. 2.3 allow us to train agents significantly faster than traditional techniques permit. The proposed approaches in these aims are complementary to each other and are also mostly orthogonal to recent algorithmic advances in this field. We believe that the aims in the proposal will result in important and new algorithms that will vastly improve state-of-the-art deep reinforcement learning and broaden the applicability to many real-world problems in science and engineering.

## 1.2 Novelty of the proposal

**Aim 1.** There have been a number of approaches attempting to improve the stability and convergence of (deep) reinforcement learning since deep Q-learning was first proposed [13, 14]. Some of those techniques have been presented in [22, 23, 24]. In this section, we will only focus on a subset of them. The original algorithm required millions of training steps to achieve human-level performance on Atari games. To improve the stability, recently, double Q-learning was combined with deep neural networks, with the goal to alleviate the overestimation issue observed in Q-learning [25, 26, 16]. The key idea is to use two Q-networks for the action selection and Q-function value calculation, respectively. The action of the target is first evaluated using the current network parameters, then the target value is computed using a set of parameters from the previous iteration. Trust region policy optimization [22] and related techniques [27, 28, 29, 30] make several approximations to guarantee improvement. Another notable advance is “prioritized replay” [31]. The idea is to increase the replay probability of experience tuples that have a high expected learning progress measured by temporal difference errors. In addition to the variants of Q-learning, other network architectures have been proposed. The dueling network architecture applies an extra network structure to learn which states are (or are not) valuable, without having to learn the effect of each action for each state [32]. A distributed version of the deep actor-critic algorithm without experience replay was evaluated recently [33]. It deploys multiple threads learning directly from current transitions. The approach is applicable to both value-based and policy-based methods, off-policy as well as on-policy methods, and in discrete as well as in continuous domains. The model-free episodic control approach evaluates state-action pairs based on episodic memory using k-nearest neighbors with hashing functions [34]. Bootstrapped deep Q-learning carries out temporally-extended (or deep) exploration, thus leading to much faster learning [35]. Our fast reward propagation differs from all of the aforementioned approaches. The key idea of our method is to propagate delayed and sparse returns back in Q-network training, and thus greatly improve the efficiency and performance. It is worth noting that our method formulates this propagation step via a constrained optimization formulation, which is completely different from the early work on  $Q(\lambda)$  and  $n$ -step Q-learning algorithms, which have been shown to be unstable when being used to train deep Q-networks for Atari games [36]. In addition, we have observed substantial and consistent improvements on Atari games over the above-mentioned approaches, thus suggesting the novelty and wide applicability of our method.

**Aim 2.** For multitask and transfer learning, the actor-mimic approach proposed a compression technique to train a single policy network that learns how to act in a set of distinct tasks by using the guidance of

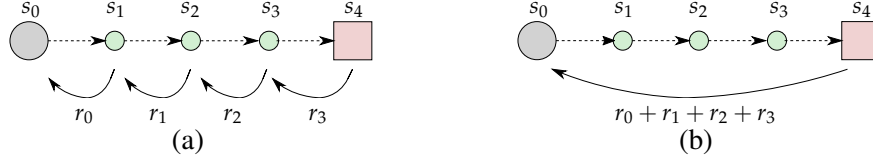


Figure 1: Fast reward propagation: In (a) we illustrate that 4 iterations are required until a reward is propagated from the target (rectangle) to the origin (circle) via intermediate states. In (b) we show that our approach requires a single iteration by choosing the entire path between  $s_0$  and  $s_5$ .

several expert teachers [37]. A very recent work [38] introduced progressive training of tasks. To the best of our knowledge, these are the most relevant work in multitask and transfer deep reinforcement learning. It is worth noting that our new shared representation approach proposed here is distinct from the actor-mimic method and progressive training. Instead of just using a single policy network or instead of using lateral connections, we decompose the Q-function neural networks into a hierarchically modular structure such that it can capture the shared decision logic and the specific visual representations and policies of related tasks. A similar modular neural network is recently posted on arXiv [39], but it is only limited for training new agent-task combinations with pretrained agent modules and task modules. Our hierarchical decomposition is more natural and more flexible than the aforementioned baselines.

**Aim 3.** Learning from human demonstrations has been widely studied in robotics. Inverse reinforcement learning approaches, including apprenticeship learning [40, 41], have been used to estimate reward functions in environments where no explicit reward functions are given. New variants, such as guided cost learning [42], have been published on improving learning from human demonstrations. Instead of considering tasks without reward functions, here we focus on the problems in the regular Markov decision process (MDP) setting with explicit numerical reward signals. It is also quite common that human guidance is hard and expensive to obtain. In many cases, only segments of human behavior, sparse human guidance on action choice, or additional reward signals can be accessed. Our goal in this aim is to extract information from sparse or limited human guidance, with which traditional Monte Carlo rollout algorithms would easily overfit. First, we propose to incorporate human trajectories into deep Q-learning via regularization. Second, we plan to apply the potential-based reward shaping scheme to transform the human guidance into a modified but more effective reward function. For both approaches, we will control the gradient backpropagation such that the sparse human guidance will only have impact on the top decision layers in deep Q-networks, and thus avoid potential overfitting. In addition, we will study the effectiveness of different types of human guidance and provide insights to further accelerate human-guided reinforcement learning on complex tasks with only small scales of computing resources.

## 2 Research Plan

This grant proposes three research goals, which will result in new reinforcement learning strategies that will vastly improve the state-of-the-art reinforcement learning tools.

### 2.1 Aim 1: Fast Reward Propagation

Q-learning algorithms optimize the cost function given in Eq. (2) and are based on the Bellman optimality equation which characterizes its stationary points. Classic implementations compute a stochastic gradient descent using one-step sequence tuples  $(s_j, a_j, r_j, s_{j+1})$ . Hence observed reward propagates one step at a time only. This immediately leads to the conclusion that a significant number of iterations are required to propagate sparse reward signals. Re-iterating the aforementioned example: an agent navigating a maze



and observing a reward only once reaching the target requires  $\mathcal{O}(|P|)$  iterations until the observed reward propagates from the target to a point. Hereby,  $|P|$  denotes the length of the shortest path from a point to the target. This effect is visualized in Fig. 1 (a) and is due to the fact that one-step sequence tuples are used for updating the parameters  $\theta$  of the Q-function. However, significantly more information, *i.e.*, entire episodes of observations  $s_j$  and actions  $a_j$ , are available in the experience replay memory. Taking advantage of this asset, we propose a new approach which significantly speeds up training, at almost no additional cost. We show the intuition for our idea in Fig. 1 (b). By considering larger sequences we are able to propagate information from an observed reward to any observation in  $\mathcal{O}(1)$  iteration compared to the  $\mathcal{O}(|P|)$  steps which are required in the classical approach. This improvement is crucial since deep Q-learning is inherently a time-consuming process which requires a significant amount of training data. Training is even more cumbersome if rewards are sparsely distributed, which is often the case for real-world environments, *e.g.*, when considering grasping or sorting tasks.

**Optimality tightening:** Here we propose a new deep Q-learning algorithm that exploits longer state-transitions in experienced replays. Please note that for notational simplicity, in this aim, we assume that the environmental dynamics are deterministic, *i.e.*, the new state and reward are solely decided by the current state and action. It is easy to show that mathematically the proposed approach will also work in stochastic environments. From the Bellman optimality equation we know that the following series of equalities hold for the optimal Q-function  $Q^*$ :

$$Q^*(s_j, a_j) = r_j + \gamma \max_a Q^*(s_{j+1}, a) = \dots = r_j + \gamma \max_a \left[ r_{j+1} + \gamma \max_{a'} \left[ r_{j+2} + \gamma \max_{\tilde{a}} Q^*(s_{j+3}, \tilde{a}) \right] \right].$$

Evaluating such a sequence exactly is not possible in a reinforcement learning setting since the enumeration of intermediate states  $s_{j+i}$  requires exponential time complexity  $\mathcal{O}(|\mathcal{A}|^i)$ . It is however possible to take advantage of the episodes available in the replay memory  $\mathcal{D}$  by noting that the following sequence of inequalities holds for the optimal action-value function  $Q^*$  (with the greedy policy), irrespective of whether the policy generating the sequence of actions  $a_j, a_{j+1}, \text{etc.}$ , which results in rewards  $r_j, r_{j+1}, \text{etc.}$  is optimal or not:

$$Q^*(s_j, a_j) = r_j + \gamma \max_a Q^*(s_{j+1}, a) \geq \dots \geq \sum_{i=0}^k \gamma^i r_{j+i} + \gamma^{k+1} \max_a Q^*(s_{j+k+1}, a) = L_{j,k}^*.$$

Note the definition of the lower bounds  $L_{j,k}^*$  for sample  $j$  and time horizon  $k$  in the aforementioned series of inequalities.

We can also use this series of inequalities to define upper bounds. To see this note that

$$Q^*(s_{j-k-1}, a_{j-k-1}) \geq \sum_{i=0}^k \gamma^i r_{j-k-1+i} + \gamma^{k+1} Q^*(s_j, a_j),$$

which follows from the definition of the lower bound by dropping the maximization over the actions, and a change of indices from  $j \rightarrow j - k - 1$ . Reformulating the inequality yields an upper bound  $U_{j,k}^*$  for sample  $j$  and time horizon  $k$  as follows:

$$U_{j,k}^* = \gamma^{-k-1} Q^*(s_{j-k-1}, a_{j-k-1}) - \sum_{i=0}^k \gamma^{i-k-1} r_{j-k-1+i} \geq Q^*(s_j, a_j).$$

In contrast to classical techniques which optimize the Bellman criterion given in Eq. (2), we propose to optimize the Bellman equation subject to constraints  $Q_\theta(s_j, a_j) \geq L_j^{\max} = \max_{k \in \{1, \dots, K\}} L_{j,k}$ , which defines the largest lower bound, and  $Q_\theta(s_j, a_j) \leq U_j^{\min} = \min_{k \in \{1, \dots, K\}} U_{j,k}$ , which specifies the smallest

upper bound. Hereby,  $L_{j,k}$  and  $U_{j,k}$  are computed using the Q-function  $Q_{\theta^-}$  rather than the unknown optimal Q-function  $Q^*$ . Hereby,  $\theta^-$  denotes a recent estimated parameter and the integer  $K$  specifies the number of future and past time steps which are considered. Also note that the target used in the Bellman equation is obtained from  $y_j = L_{j,0} = r_j + \gamma \max_a Q_{\theta^-}(s_{j+1}, a)$ . In this way, we ignore the dependence of the bounds and the target on the parameter  $\theta$  to stabilize the training. Taking all the aforementioned definitions into account, we propose the following program for reinforcement learning tasks:

$$\min_{\theta} \sum_{(s_j, a_j, r_{j+1}) \in \mathcal{B}} (Q_{\theta}(s_j, a_j) - y_j)^2 \quad \text{s.t.} \quad \begin{cases} Q_{\theta}(s_j, a_j) \geq L_j^{\max} & \forall (s_j, a_j) \in \mathcal{B} \\ Q_{\theta}(s_j, a_j) \leq U_j^{\min} & \forall (s_j, a_j) \in \mathcal{B} \end{cases} \quad (3)$$

This program differs from the classical approach given in Eq. (2) via the constraints, which is crucial. Intuitively, the constraints encourage faster reward propagation as we show next. In turn, faster reward propagation result in tremendously better performance as our preliminary results indicate.

Before doing so we describe our current technique to optimize the constrained program in Eq. (3) more carefully. The cost function is generally non-convex in the parameters  $\theta$ , and so are the constraints. We therefore make use of a quadratic penalty method to reformulate the program into

$$\min_{\theta} \sum_{(s_j, a_j, r_{j+1}) \in \mathcal{B}} \left[ (Q_{\theta}(s_j, a_j) - y_j)^2 + \lambda (L_j^{\max} - Q_{\theta}(s_j, a_j))_+^2 + \lambda (Q_{\theta}(s_j, a_j) - U_j^{\min})_+^2 \right], \quad (4)$$

where  $\lambda$  is a penalty coefficient and  $(x)_+ = \max(0, x)$  is the rectifier function. Augmenting the cost function with  $\lambda (L_j^{\max} - Q_{\theta}(s_j, a_j))_+^2$  and/or  $\lambda (Q_{\theta}(s_j, a_j) - U_j^{\min})_+^2$  results in a penalty whenever any optimality bounding constraint gets violated. The quadratic penalty function is chosen for simplicity. The penalty coefficient  $\lambda$  can be set as a large positive value or adjusted in an annealing scheme during training. We can optimize this cost function with stochastic (sub-)gradient descent using an experience replay memory from which we randomly draw samples, as well as their successors and predecessors. We emphasize that the derivatives correcting the prediction of  $Q(s_j, a_j)$  not only depend on the Q-function from the immediately successive time step  $Q(s_{j+1}, a)$  stored in the experience replay memory, but also on more distant time instances if constraints are violated. Our proposed formulation and the resulting optimization technique hence encourage faster reward propagation, and the number of time steps depends on the constant  $K$  and the quality of the current Q-function. For stochastic MDPs, we can still formulate a similar constrained optimization problem but with expectation constraints. After applying the quadratic penalty method, one can use Jensen's inequality to show that Eq. (4) is an empirical upper bound of the the penalty loss function in the stochastic setting, thus ensuring that the optimality tightening also works approximately well.

The computational complexity of the proposed approach increases with the number of considered time steps  $K$ , since additional forward passes are required to compute the bounds  $L_j^{\max}$  and  $U_j^{\min}$ . However, we can increase the memory size on the GPU to compute both the bounds and targets in a single forward pass if  $K$  is not too large. If at all a problem, we can further alleviate this increase by randomly sampling a subset of the constraints rather than exhaustively using all of them. More informed strategies regarding the choice of constraints are possible as well since we may expect lower bounds in the more distant future to have a larger impact early in the training. In contrast once the algorithm is almost converged we may expect lower bounds close to the considered time-step to have bigger impact.

To efficiently compute the discounted reward over multiple time steps we add a new element to the experience replay structure. Specifically, in addition to state, action, reward and next state for time-step  $j$ , we also store the real discounted return  $R_j$  which is the discounted cumulative return achieved by the agent in its game episode.  $R_j$  is computed via  $R_j = \sum_{\tau=j}^T \gamma^{\tau-j} r_{\tau}$ , where  $T$  is the end of the episode and  $\gamma$  is the discount factor.  $R_j$  is then inserted in the replay memory after the termination of the current episode or after reaching the limit of steps. All in all, the structure of our experience replay memory consists of tuples of the form  $(s_j, a_j, r_j, R_j, s_{j+1})$ .



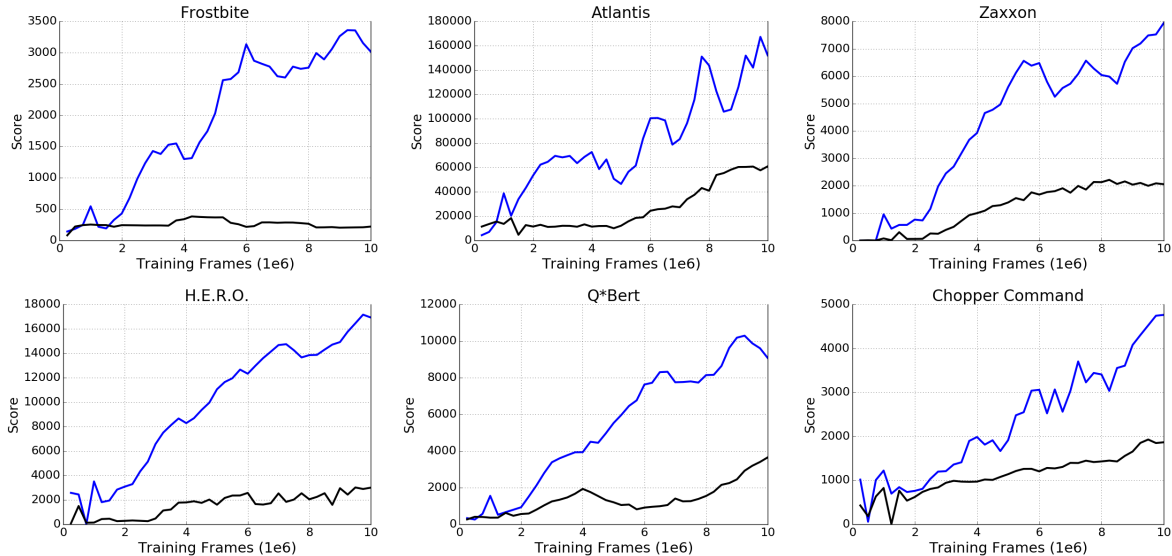


Figure 2: Game scores for our algorithm (blue) and DQN (black) using 10M training frames.

**Preliminary Results:** In the following we present preliminary results of the proposed strategy, evaluated on the Arcade Learning Environment benchmark [43]. We evaluated our first implementation of the fast reward propagation algorithm on 49 Atari 2600 games. We adopted the same neural network Q-function structure and experimental setting as suggested by [14].

On the majority of the Atari games, we observed that our fast reward propagation algorithm significantly outperforms the original deep Q-learning algorithm (data not shown due to space limit). Fig. 2 shows a comparison on several examples. Our algorithm converged much faster than the deep Q-learning algorithm and achieved human-level performance on many games with only a small number of frames (usually one magnitude less), compared to hundreds of millions reported by other existing methods [14, 31, 32]. In contrast to the original DQN algorithm, which took more than 10 days to obtain 241% mean and 93.5% median of normalized scores compared to human, our method only requires less than 1 day to achieve 345.7% mean and 105.7% median, on 49 Atari games reported in [14].

**Future directions:** Besides evaluating our approach on a larger set of benchmarks, we plan to first integrate the strategy with existing deep reinforcement learning approaches such as double deep Q-learning [26, 16] and actor-critic algorithms *etc.* Questions regarding a good choice of penalty function and a good choice of the penalty coefficients need to be answered. At the moment we use a quadratic penalty function and a constant penalty coefficient  $\lambda$  identical for both bounds. However, more complex penalty functions and sophisticated optimization approaches may yield even better results than the ones we reported.

Another important direction is to consider structured actions as macro actions in the current deep Q-learning algorithms [44]. Macro actions allow the agent to make a short sequence of actions for faster run-time decisions. We will explore approaches to discover high-reward structured patterns of actions from experienced replays. This approach will be similar to option learning and temporal abstraction approaches. Instead of using a predefined semi-Markov or hierarchical structure, we hope to directly learn effective macro actions with the goal of improving the throughput of real-time decisions. To this end we again construct informative constraints for optimality tightening when reward signals are delayed. As before we plan to evaluate our extensions on the Arcade Learning Environment and additionally on tasks available in the standard OpenAI Gym [45], the Mujoco physics engine [46] and driving simulators such as TORCS [47].

**Potential pitfalls.** Throughout the aforementioned discussion we assumed the action space to be discrete. It is unclear how optimality tightening performs on continuous action spaces. However many real world

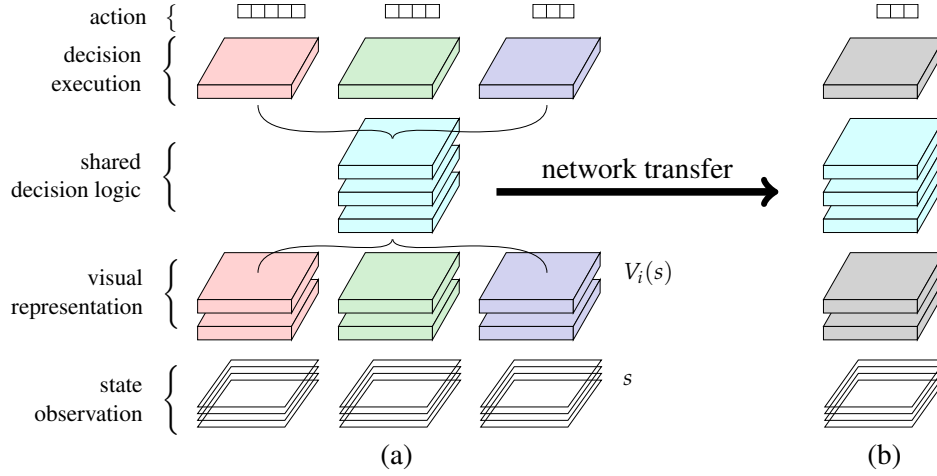


Figure 3: Shared multitask architecture. In (a) we visualize the training process where the visual representation layer, the shared decision logic and the decision execution are trained jointly for multiple games. In (b) a new environment is obtained and only the visual representation and the decision execution are learned while the decision logic is transferred.

problems such as autonomous driving, grasping and sorting possess continuous action spaces. Two obvious solutions are to either discretize these spaces and apply the aforementioned algorithms, or discretize iteratively to refine the action space at locations which are likely important. However neither of those does address the original and important problem of continuous action spaces. In spirit similar to the aforementioned discussion and contrasting existing work such as [48, 49] we want to extend our aforementioned method to continuous action spaces. A key challenge in this direction is the maximization of action which cannot be solved exactly in general. We will explore two recently proposed continuous control algorithms, the deep deterministic policy gradient (DDPG) [48] and the normalized advantage function (NAF) [49], to test the effectiveness of the optimality tightening. In DDPG, the maximization of action is approximated by a neural network, while in NAF, the maximization is exact with the proposed quadratic form of advantage function.

## 2.2 Aim 2: Effective Information Sharing

Besides propagating rewards faster, we suggest to take advantage of the fact that many real-world tasks are similar to one another. Consider, for example, driving a car and navigating an SUV or a larger truck. A similar set of skills is required to succeed in both. However, despite early evidence of benefits for sharing information in the reinforcement learning literature [50], sharing of skills is rarely encoded in deep reinforcement learning techniques these days. Therefore we propose an approach for sharing representations across reinforcement learning tasks. In the following we differentiate between multitask training which is visualized in Fig. 3 (a) and a corresponding transfer learning setting, which is schematically illustrated in Fig. 3 (b).

Before discussing the details for both approaches, we note that there is significant very recent evidence across different disciplines that sharing of information across multiple tasks increases the performance of individual methods, sometimes significantly. Consider for example approaches in computer vision such as [51, 52], or techniques in machine translation [53]. We expect similar results when considering autonomous agents and describe our methods for achieving those in the following.

**Multitask learning:** The environment state representations often differ between tasks. Similarly, the decision and the number of possible actions and the inherent reward mechanisms differ from task to task. However, it is natural to assume a shared decision logic across similar tasks. We therefore espouse a network architecture as visualized in Fig. 3 (a) to yield very meaningful results. There are three stacks of neural

network layers in this architecture. The lower stack of the Q-function network close to the state observation, *e.g.*, images in Fig. 3 (a), is designed to extract a common (visual) representation. Those network components are task specific because the visual input often differs from task to task, particularly when considering the Arcade Learning Environment. However, relevant or similar tasks tend to have similar semantics of their environments. After extracting a common visual representation we use a series of shared layers to transform an extracted high-level visual representation into an abstraction for the desired decision. The decision is then made in a task-specific layer on top of the shared decision logic abstraction. A task-specific layer is necessary because the number of available actions differs from task to task.

For the effectiveness of the proposed architecture we need to ensure that the visual representation provided as input to the shared decision logic layers is reasonably similar. Augmenting the traditional Q-learning cost function with moment-matching constraints provides a good technique to ensure flexibility between entirely different games while encouraging consistency between related tasks.

To describe the approach more formally, let us introduce some additional notation: we use  $F_i(s)$  to refer to the visual representation obtained from state  $s$  in task  $i$ . See Fig. 3 (a) for an illustration. In addition to the regular Q-learning loss, or the one from our novel technique proposed in Sec. 2.1, we use a moment-matching based regularization, where the expectation or moments over the visual representations for task  $k$ , *i.e.*,  $\mathbb{E}[F_k(s)]$ , is compared to those of any another task  $m$ . Note that here the expectation can be replaced by a nonlinear kernel mapping. Hereby the expectation is taken w.r.t. the states in the experience replay memory or a subset of selected samples in the minibatch. We include this difference of expectations, *i.e.*, a moment-matching property, into the cost function by considering pairs of tasks  $k, m$  as follows:

$$\sum_{(s_j^m, a_j^m, r_j^m, s_{j+1}^m) \in \mathcal{B}^m} \left( Q_{\theta_m}^m(s_j^m, a_j^m) - y_j^m \right)^2 + \sum_{(s_j^k, a_j^k, r_j^k, s_{j+1}^k) \in \mathcal{B}^k} \left( Q_{\theta_k}^k(s_j^k, a_j^k) - y_j^k \right)^2 + \lambda \|\mathbb{E}[F_k(s)] - \mathbb{E}[F_m(s)]\|_2^2,$$

where  $\lambda$  is a hyper-parameter to trade between the Bellman equation, respectively our modification described in Sec. 2.1, and the additional regularization term.

We envision this technique to be beneficial and effective for two reasons. Firstly, by jointly training the decision for multiple tasks, we are able to increase the amount of training data drastically, which permits training of larger networks. Secondly, we are able to exploit correlations between different tasks. Hence we can take advantage of information available in multiple datasets and let the learning mechanism decide which representations are valuable. We envision this to be particularly useful in the reinforcement learning realm.

A variety of tasks have been proposed for multi-task reinforcement learning [54, 55, 56, 57, 58, 59, 60]. They differ from our proposed approach in multiple aspects. Firstly, all attempts that share abstract data representations extracted from environment states using deep nets are still at its infancy, particularly when considering reinforcement learning tasks. Secondly, none of the proposed approaches discusses an architecture similar to the one we propose in Fig. 3 (a).

**Transfer learning:** In addition to the aforementioned advantages of jointly training multiple tasks, there are also benefits of the proposed architecture when targeting novel environments. A possible concept which we want to explore is shown in Fig. 3 (b) and illustrates that we leverage the shared decision logic trained via multiple tasks.

Taking advantage of transferring a shared decision logic reduces the number of trainable parameters while making use of the information available through previously trained layers. This encourages faster training with fewer data. To ensure the success of this concept we require the extracted visual representation  $F(s_t)$  of the newly obtained task to be close to the expectations of previously trained tasks. To achieve this goal, we can again add a regularization term to the cost function:

$$\sum_{(s_j, a_j, r_j, s_{j+1}) \in \mathcal{B}} \left( Q_{\theta}(s_j, a_j) - y_j \right)^2 + \lambda \sum_k \|\mathbb{E}[F(s)] - \mathbb{E}[F_k(s)]\|_2^2,$$

where  $\mathbb{E}[F_k(s)]$  are the moments of visual representations retained when training previous tasks with our multitask deep Q-learning architecture. Similar procedures have been recently considered for tasks other than reinforcement learning [21]. It is again worth noting that this proposed method can be combined with our previously proposed ‘fast reward propagation’ method as well as other recently proposed techniques such as double deep Q-learning.

We will evaluate the proposed procedure using a cross-validation technique over games in the Arcade Learning Environment [43]. More specifically, we will train one large multitask network on similar Atari games, while leaving out another game for testing our transfer learning procedure. We will first evaluate the performance of joint training *vs.* standalone training of similar games. We will then train the visual extractor and the decision network for the left-out game and compare the training time and the number of necessary episodes to the standalone Q-learning approach. In this way we can demonstrate the effectiveness of the proposed approaches by sharing of representations of multiple tasks and the generalization ability to new tasks.

Excellent surveys for the transfer learning setting in the reinforcement learning domain have been presented in [61, 62]. We refer the interested reader to those surveys and references therein for additional details. More recent work such as [63, 64, 65] considers the online setting for tabular approaches and linear function approximators [63], learning of shaping functions in embedding spaces [64] and Multi-agent Object-Oriented MDPs [65]. The work of [64] is in spirit similar to what we suggest in this proposal. However, contrasting the work of Konidaris *et al.* [64] our approach is based on a deep net architecture, direct parameter transfer rather than reward shaping transfer, and an additional regularization term.

**Future directions:** Besides carefully evaluating the proposed cost functions designed for effective information sharing, we plan to combine them with fast reward propagation as well as techniques such as prioritized replay [31]. We also envision our proposed methods to be useful for combinations of planning based algorithms and Q-learning [66], large action spaces and macro-actions [67, 44], as well as parameterized continuous action spaces [68] and Q-networks with memory [69] that we plan to address in future work. In addition, we also plan to investigate effective mechanisms of sharing the decision logic in the temporal abstraction context (or in semi-MDPs).

Besides the discussed algorithmic advances that we will investigate, we also plan to address vision based tasks such as navigation in 3D indoor environments. Instead of just extracting abstract representations directly from images we will augment deep reinforcement learning architectures with tools such as support surface prediction [70], indoor layout estimation [71, 72] or semantic segmentation, *e.g.*, [73].

**Potential pitfalls:** The current proposed architecture requires different tasks to be similar to allow efficient network information sharing. For tasks without enough similarity, it is easy to see that the proposed method may not work because their visual representations and decision logic may differ significantly. To address this issue, we will design systematic experiments to check whether two tasks are similar enough to allow efficient information sharing. Nevertheless, our approach should be able to be applied for multiple tasks within the same visual environment, which is ubiquitous in many robotics applications.

### 2.3 Aim 3: Learning from Human Guidance

In Sec. 2.1 we proposed an effective algorithmic mechanism for propagating rewards efficiently by considering longer chains that are readily available within the replay memory when the reward signal is sparse and delayed. A human teacher, on the other hand, provides direct feedback by supplying either guidance of actions or reward/punishment signals as extra input to the current reward function. For example, in learning from demonstration (or apprenticeship learning) [40, 41, 74, 75, 76], a policy is trained from examples including demonstrations, trajectories or real-time feedback provided by a human teacher. Normally, such examples are stored as sequences or episodes of state-action pairs. This can be particularly effective when agents face challenging tasks in complex environments where the reward function is unknown or noisy,

such as learning Go game playing and learning locomotive movements [77, 78, 79, 80, 42, 81, 82, 83, 84]. However, a large training set of demonstrations is often required for learning a good reward function and/or a good policy. Different from previous work, we hope to exploit human guidance to accelerate deep reinforcement learning algorithms when only a small set of demonstration examples are provided. Note that we assume the reward signal to be provided, and the accessibility of human guidance to be expensive, which is generally the case for many RL problems. To learn from human guidance, we propose to develop novel algorithms which extract information from demonstrated examples and enable better learning performance. There are two scenarios where humans can assist reinforcement learning agents: 1) humans can provide full trajectories as demonstrations for agents; 2) humans can provide online feedback to agents, including augmenting reward and providing guidance of actions. It is also critical to understand in which scenario human input will enable more effective guidance for training of agents, potentially providing further insights for effective human-robot interactions. Here we propose two ways to integrate human guidance and improve deep Q-learning.

**Learning Q-function with human demonstrations:** One possible way to exploit complete human trajectories is to apply the Monte Carlo rollout learning and train value functions in a supervised manner. As discussed above, deep neural networks are often used to obtain abstract representations from environment states in complex tasks. Therefore, it is not practically feasible to learn a good Q-function directly from expensive and limited human trajectories, as sample complexity is usually prohibitive for deep neural networks. Furthermore, we can only obtain short fragments of human trajectories, thus making Monte Carlo rollout not applicable.

To handle the sparsity of human demonstrations, we propose a novel Q-learning algorithm that treats human demonstrations as soft constraints to augment the signal in training. Intuitively, when a state  $s_t$  is observed in a training step, we should be able to mimic the corresponding human action if the same state can be found in human demonstrations. Even when no identical state appears in human demonstrations, we still hope to take similar actions that a human took in a similar state. In addition, since we also observe rewards within human trajectories, we hope to trust actions that lead to high future reward. Formally, for each  $e_j = (s_j^h, a_j^h, r_j^h, s_{j+1}^h)$  tuple in an episode  $e$  with return  $R_e$ , we develop new regularization functions which augment the original Q-learning via:

$$L_1^h(\theta) = - \sum_{e_j} w(R_e) \log \frac{\exp(\alpha Q_\theta(s_j^h, a_j^h))}{\sum_a \exp(\alpha Q_\theta(s_j^h, a))} \quad \text{or} \quad L_2^h(\theta) = \sum_{e_j} w(R_e) (Q_\theta(s_j^h, a_j^h) - \max_a Q_\theta(s_j^h, a))^2_+,$$

where  $\alpha$  is a temperature parameter in the Boltzmann distribution;  $w(R_e)$  is a weight function to emphasize transitions in high-reward episodes. Both regularization functions  $L^h$  encourage the Q-function to choose the human action  $a_j^h$  over other actions in state  $s_j^h$ . It is not hard to see that we can augment the original Q-learning with such regularization functions and perform a joint optimization. A hyper-parameter can be introduced to adjust the trade-off between the Bellman residual error and the human guidance. Stochastic gradient descent algorithms are readily applicable for learning the parameters of the Q-function. Instead of optimizing all tuples from human demonstrations simultaneously, we will also subsample a mini-batch of tuples. A key question here is whether we want to draw human tuples uniformly or in a biased way. For example, we can couple a subset of human tuples with states that are similar to the states in the tuples used for minimizing the Bellman residual error. As a result, human guidance will immediately enhance the signal of action choices in Q-learning. Similar to the prioritized experience replay, importance weights will be considered to reduce sampling bias and variance. The uniform sampling will ensure the unbiased estimation of gradients from human demonstrations, while the biased approach may provide faster convergence. Also, since the human guidance might be sparse, it is risky to perform the full back-propagation gradient descent on human tuples. Here we will only allow decision-layer gradients on human tuples. Extensive experiments will be performed to evaluate the detail choices of regularization functions and sampling schemes.



Furthermore, we plan to explore the possibility of combining optimality tightening (see Aim 1) and human-guided Q-learning for further improvements. Although in the formulation of optimality tightening, we hope to identify the feasible regions of Q-function across all possible state-action pairs, it is not necessary to treat all state-action pairs identically. With a reasonable Q-function, the trajectories generated will mainly focus on frequently visited or high-reward states. If we choose the second margin-based regularization function  $L_2^h$  for human-guided Q-learning, it is easy to see that its functional form is identical to the penalty function we proposed for optimality tightening. Here we will also explore other alternative approaches. For example, we will use human tuples to select informative upper and lower bounds for any given training tuple when using optimality tightening. Similar to prioritized sweeping, we hope to constrain the Q-function such that it favors these high-reward human actions in similar states. Moreover, we can use the optimality bounds to identify informative human tuples for guiding the Q-learning.

**Learning reward shaping from human guidance:** Due the sparse and delayed nature of reward functions, reinforcement learning algorithms employ iterative approaches by back propagating the future reward signal to the current state value function, which is often time consuming. Reward shaping, which assigns additional numerical rewards to guide actions, has been proven very successful for enhancing the convergence rate of conventional reinforcement learning algorithms. Potential-based reward shaping modifies the reward function in that  $r^{new}(s_t, a_t) = r^{original}(s_t, a_t) + \gamma P(s_{t+1}) - P(s_t)$ , where  $P$  is a potential function providing additional information in reward [85, 86, 87, 88, 89, 90, 91]. [85] proved that the potential-based reshaping maintains the same optimal policy as the one we can obtain from the original reward function. It is important that we will need such potential functions to avoid possible policy biases by directly included additional reward/punishment signals in the reward function. However, a major problem with reward shaping is that often we are not able to obtain any informative potential function. Here we propose to exploit human guidance to construct effective reward shaping potentials. Existing approaches of reward shaping from human guidance are mostly based on inverse reinforcement learning (IRL) [92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106], which requires a huge amount of human data in training. In the traditional reinforcement setting, IRL works reasonably well because the Q-functions are often very simple, tabular or linear. When deep Q-networks are applied, it is very difficult to avoid overfitting on human demonstrations. A potential way to alleviate this issue is to fix the reward shaping potential before Q-learning. Here we propose to develop a joint training method of reward shaping and Q-networks. Instead of using another deep neural network for the reward shaping potential, we use a simple linear function that takes information from a subset of layers of the Q-network as input, with the goal to avoid overfitting. Thus the parameters of visual representations and the lower decision layers will be shared between the Q-network and the shaping potential  $P$ . To incorporate the human guidance into the potential function, we will explore methods for different types of human guidance data. First, if we have the complete human trajectories or demonstration episodes, we can directly let  $P$  to regress the final return. In other cases, where we can only obtain sparse human guidance, we will train  $P$  to mimic a decision function such as one similar to the  $L_2^h$  function, or let  $P$  fit the additional reward/punishment signal. Note that the reward shaping scheme is very different from the joint Q-learning proposed in the previous sub-aim in that it can handle various types of sparse human guidance in a flexible way.

**Evaluation Plan:** To evaluate the algorithms proposed in this aim, we need to obtain high-quality human demonstrations. We plan to use crowdsourcing platforms and encourage students to get familiar with specific tasks and generate human expert trajectories. Specifically, we will ask humans to generate three types of guidance data. First, a dataset of complete trajectories will be generated offline. These trajectories will be applied for Q-learning and reward shaping. In addition, we can also test whether the current deep Q-learning algorithms can be improved by inserting these trajectories into the replay memory. Second, real-time feedback, such as action corrections and/or extra reward and punishment signals, will be generated during simulations interactively. With real-time feedback data, we will test the effectiveness of reward shaping for speeding up deep Q-learning. As preliminary studies, we will first evaluate our proposed methods



on the Atari games and the TORCS driving simulator. In addition, extensive experimental comparisons will also be performed to understand the effectiveness of different types of human guidance.

**Potential pitfalls:** A potential caveat of our proposed methods is that it is not clear how to generate effective human guidance to assist deep reinforcement learning, especially for tasks that ask for long-term goals. To address these issues, we plan to develop an active learning approach that identifies transitions or states that the trained deep Q-network has difficulty making a clear decision. This approach takes the the Q-network decisions as input and quantifies whether human guidance is needed. Entropy or other uncertainty metrics can be applied here. This procedure can be coupled with the Q-learning to obtain high-quality human guidance in an interactive way.

### 3 Broader Impact

**Promoting Education.** The PIs will incorporate demos from the project into the Artificial Intelligence, Machine Learning, Computer Vision, and Image Processing courses they regularly teach. These courses reach enrollments between 100 and 200 undergraduate and graduate students from diverse backgrounds, i.e., CS and ECE to civil engineering. We will integrate the obtained material in upper-level undergraduate courses – computer vision and artificial intelligence – which attract a more diverse set of students than typical computer science courses. A long-term goal of the educational plan is the development of a new reinforcement learning undergraduate course to train the next generation of computer scientists, electrical engineers and other researchers in the use of these tools for their research and applications.

**Undergraduate Research and Outreach.** The PIs have identified a number of undergraduate research opportunities within the research plan: 1) implementing optimality tightening techniques and performing extensive experiments for reinforcement learning tasks in stochastic environments; 2) exploring information sharing of multiple tasks in the same environment; 3) exploring options of generating human guidance for deep Q-learning. Each task will be formulated as self-contained projects to be performed by undergraduate students in course projects or via independent studies. The PIs and graduate students will mentor students and help them writing scientific reports. In the future, the PIs will also seek research experience for undergraduates (REU) opportunities to support students from minority groups and/or non-research universities.

In addition, the PIs will host demos and discussion sessions at CS@Illinois events targeted at female middle and high school students, such as ChicTech, an initiative organized to generate interest in computer science among Illinois high school girls, and the G.A.M.E.S (Girls Adventures in Mathematics, Engineering, and Science) Computer Science camp.

**Societal Impact.** The proposed approaches have the potential to enable applications with high societal impact. Deep reinforcement learning techniques have been used for electricity control in data centers, autonomous driving, sensing and locomotion in robotics, and various applications in operations research. In the shorter term, we will increase the impact and visibility of our work in multiple ways. The proposed research plan will expedite various scientific and engineering applications by establishing novel efficient reinforcement learning algorithms. The educational promotion will also support undergraduate training in research and promoting diversity in the workforce.

**Broad Dissemination.** The PIs will make all their code and data publicly available on their respective websites as well as a website devoted to this project. Additional dissemination will occur through peer-reviewed publications in major conferences and journals as well as presentations at conferences such as the Conference on Neural Information Processing Systems (NIPS), the International Conference on Machine Learning (ICML), and the International Conference on Learning Representations (ICLR). In addition, the

PIs will further distribute the results of this work via other channels, including incorporating into the course materials, broadcasting of the work through university press releases, and participating data and innovation challenges issued by public, non-profit and other organizations in the community.

## **4 Results from Prior NSF Support**

The PIs commenced their appointments as assistant professors at the University of Illinois at Urbana- Champaign in 2015 and 2016, respectively, and have not previously received NSF support.

## **5 Collaboration Plan**

### **Resources and Project Team**

To perform the research proposed in this grant, we will utilize the existing computational resources in our research groups and get computing support from the University of Illinois at Urbana-Champaign (UIUC) and the National Center of Supercomputing and Applications (NCSA). We have GPU-based workstations with 11 NVIDIA Titan-X 12GB graphics cards, 512GB RAM, 10TB Storage and advanced Intel Xeon CPUs. In the BlueWaters from NCSA, we have access to more than 4000 GPU cards and 300000 CPUs via a NCSA faculty fellowship to the PI (Peng). Our team consists of an established machine learning researcher with a strong record of excellence in graphical models, optimization, large datasets with applications to genomics (Peng), and a computer vision expert with an established record of excellence in machine learning and advanced optimization applications in computer vision (Schwing). The PIs are ideally situated to complete the proposed work, as they are experts in scene understanding, image synthesis, optimization and inference algorithms. Recently, the PIs have prepared and submitted multiple manuscripts including some of the initial results from Aim 1 and Aim 2.

### **Project Management and Timeline**

Each PI will advise one graduate student through the three years of the grant. Both PIs are committed to building a team focused on designing effective reinforcement learning techniques and developing efficient solutions for high-impact tasks.

A timeline of our proposed efforts is as follows: In the first year we focus on our first aim. In the second year we will take advantage of the obtained results and design, evaluate and present the results related to effective information sharing. Following accompanying initial assessments during the first two years we will focus on the third aim of exploiting human guidance for deep Q-learning in the third year.

Coordination and interaction between the PIs will be easy since both PIs are physically at the same institution. We will hold a weekly in-person meeting to facilitate collaboration on the project. Students working on the project will have an additional weekly meeting to discuss specific details surrounding implementation of the various proposed project modules. In addition, students will prepare internal web pages to share the results and visualizations of their efforts as well as documenting their weekly progress and action items via a shared Google Doc devoted to the project. A shared code repository will be created for the project so that libraries and implementations can be shared across the project and released to the public for use.

## References

- [1] Y. LeCun, Y. Bengio, and G. E. Hinton. Deep learning. *Nature*, 2015.
- [2] A. Krizhevsky, I. Sutskever, , and G. E. Hinton. Imagenet classification with deep convolutional neural networks. In *Proc. NIPS*, 2012.
- [3] I. Sutskever, O. Vinyals, and Q. V. Le. Sequence to sequence learning with neural networks. In *Proc. NIPS*, 2014.
- [4] G. E. Hinton, L. Deng, D. Yu, G. E. Dahl, A.-R. Mohamed, N. Jaitly, A. Senior, V. Vanhoucke, P. Nguyen, T. N. Sainath, and B. Kingsbury. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. *IEEE Signal Processing Magazine*, 2012.
- [5] Y. Bengio, A. Courville, and P. Vincent. Representation Learning: A Review and New Perspectives. *PAMI*, 2013.
- [6] D. P. Bertsekas and J. N. Tsitsiklis. *Neuro-Dynamic Programming*. Athena Scientific, 1996.
- [7] W. P. Powell. *Approximate Dynamic Programming*. Wiley, 2011.
- [8] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 1998.
- [9] L. P. Kaelbling, M. L. Littman, and A. W. Moore. Reinforcement learning: A survey. *JMLR*, 1996.
- [10] C. J. C. H. Watkins. *Learning from delayed rewards*. PhD thesis, University of Cambridge England, 1989.
- [11] C. J. C. H. Watkins and P. Dayan. Q-learning. *Machine Learning*, 1992.
- [12] J. N. Tsitsiklis and B. Van Roy. An analysis of temporal-difference learning with function approximation. 1997.
- [13] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller. Playing Atari with Deep Reinforcement Learning. In *NIPS Deep Learning Workshop*, 2013.
- [14] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 2015.
- [15] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis. Mastering the game of Go with deep neural networks and tree search. *Nature*, 2016.
- [16] H. van Hasselt, A. Guez, and D. Silver. Deep Reinforcement Learning with Double Q-learning. In <https://arxiv.org/abs/1509.06461>, 2015.
- [17] M. Riedmiller. Neural fitted Q iteration - first experiences with a data efficient neural reinforcement learning method. In *Proc. ECML*, 2005.
- [18] S. Lange and M. Riedmiller. Deep auto-encoder neural networks in reinforcement learning. In *Proc. Int. Jt. Conf. Neural. Netw.*, 2010.

- [19] L.-J. Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 1992.
- [20] P. Wawrzynski. Real-time reinforcement learning by sequential actor-critics and experience replay. *Neural Networks*, 2009.
- [21] Z. Li and D. Hoiem. Learning without Forgetting. In *Proc. ECCV*, 2016.
- [22] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel. Trust Region Policy Optimization. In <https://arxiv.org/abs/1502.05477>, 2015.
- [23] B. C. Stadie, S. Levine, and P. Abbeel. Incentivizing Exploration In Reinforcement Learning With Deep Predictive Models. In <https://arxiv.org/abs/1507.00814>, 2015.
- [24] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel. High-Dimensional Continuous Control Using Generalized Advantage Estimation. In <https://arxiv.org/abs/1506.02438>, 2015.
- [25] S. Thrun and A. Schwartz. Issues in using function approximation for reinforcement learning. In *Proc. Connectionist Models Summer School*, 1993.
- [26] H. van Hasselt. Double Q-learning. In *Proc. NIPS*, 2010.
- [27] J. A. Bagnell and J. Schneider. Covariant policy search. In *Proc. IJCAI*, 2003.
- [28] J. Peters and S. Schaal. Reinforcement learning of motor skills with policy gradients. In *Neural Networks*, 2008.
- [29] J. Peters and S. Schaal. Natural actor-critic. In *Neurocomputing*, 2008.
- [30] J. Peters, K. Mülling, and Y. Altün. Relative entropy policy search. In *Proc. AAAI*, 2010.
- [31] T. Schaul, J. Quan, I. Antonoglou, and D. Silver. Prioritized Experience Replay. In *Proc. ICLR*, 2016.
- [32] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas. Dueling Network Architectures for Deep Reinforcement Learning. In <https://arxiv.org/abs/1511.06581>, 2015.
- [33] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu. Asynchronous Methods for Deep Reinforcement Learning. In <https://arxiv.org/abs/1602.01783>, 2016.
- [34] C. Blundell, B. Uria, A. Pritzel, Y. Li, A. Ruderman, J. Z. Leibo, J. Rae, D. Wierstra, and D. Hassabis. Model-Free Episodic Control. In <http://arxiv.org/pdf/1606.04460v1.pdf>, 2016.
- [35] I. Osband, C. Blundell, A. Pritzel, and B. Van Roy. Deep Exploration via Bootstrapped DQN. In <http://arxiv.org/abs/1602.04621>, 2016.
- [36] Rémi Munos, Tom Stepleton, Anna Harutyunyan, and Marc G Bellemare. Safe and efficient off-policy reinforcement learning. *arXiv preprint arXiv:1606.02647*, 2016.
- [37] E. Parisotto, J. L. Ba, and R. Salakhutdinov. Actor-Mimic: Deep Multitask and Transfer Reinforcement Learning. In <https://arxiv.org/abs/1511.06342>, 2015.
- [38] A. A. Rusu, N. C. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, and R. Hadsell. Progressive Neural Networks. In <https://arxiv.org/abs/1606.04671>, 2016.

- [39] C. Devin, A. Gupta, T. Darrell, P. Abbeel, and S. Levine. Learning Modular Neural Network Policies for Multi-Task and Multi-Robot Transfer. In <https://arxiv.org/pdf/1609.07088.pdf>, 2016.
- [40] P. Abbeel and A. Y. Ng. Apprenticeship Learning via Inverse Reinforcement Learning. In *Proc. ICML*, 2004.
- [41] G. Neu and C. Szepesvári. Apprenticeship Learning using Inverse Reinforcement Learning and Gradient Methods. In *Proc. UAI*, 2007.
- [42] C. Finn, S. Levine, and P. Abbeel. Guided Cost Learning: Deep Inverse Optimal Control via Policy Optimization. In <https://arxiv.org/abs/1603.00448>, 2016.
- [43] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling. The arcade learning environment: An evaluation platform for general agents. *J. of Artificial Intelligence Research*, 2013.
- [44] A. McGovern and R. S. Sutton. MacroActions in Reinforcement Learning An Empirical Analysis. Technical report, University of Massachusetts Amherst, 1998.
- [45] Openai gym. <https://gym.openai.com>. Accessed: 2016-11-16.
- [46] E. Todorov, T. Erez, and Y. Tassa. MuJoCo: A physics engine for model-based control. In *Proc. IROS*, 2012.
- [47] B. Wymann, C. Dimitrakakis, A. Sumnery, and C. Guionneau. TORCS: The open racing car simulator. 2015.
- [48] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. In <https://arxiv.org/abs/1509.02971>, 2015.
- [49] S. Gu, T. Lillicrap, I. Sutskever, and S. Levine. Continuous Deep Q-Learning with Model-based Acceleration. In <https://arxiv.org/pdf/1603.00748.pdf>, 2016.
- [50] M. Tan. Multi-Agent Reinforcement Learning: Independent vs. Cooperative Agents. In *Proc. ICML*, 1993.
- [51] L. Castrejón, Y. Aytar, C. Vondrick, H. Pirsiavash, and A. Torralba. Learning Aligned Cross-Modal Representations from Weakly Aligned Data. In *Proc. CVPR*, 2016.
- [52] I. Kokkinos. UberNet: Training a ‘Universal’ Convolutional Neural Network for Low-, Mid-, and High-Level Vision using Diverse Datasets and Limited Memory. In <https://arxiv.org/abs/1609.02132>, 2016.
- [53] O. Firat, K. Cho, and Y. Bengio. Multi-Way, Multilingual Neural Machine Translation with a Shared Attention Mechanism. In *Proc. NAACL*, 2016.
- [54] A. Wilson, A. Fern, S. Ray, and P. Tadepalli. Multi-Task Reinforcement Learning: A Hierarchical Bayesian Approach. In *Proc. ICML*, 2007.
- [55] D. Borsa, T. Graepel, and J. Shawe-Taylor. Learning Shared Representations in Multi-task Reinforcement Learning. In <https://arxiv.org/abs/1603.02041>, 2016.
- [56] A. Lazaric and M. Ghavamzadeh. Bayesian Multi-Task Reinforcement Learning. In *Proc. ICML*, 2010.



- [57] H. Li, X. Liao, and L. Carin. Multi-task Reinforcement Learning in Partially Observable Stochastic Environments. *JMLR*, 2009.
- [58] D. Calandriello, A. Lazaric, and M. Restelli. Sparse Multi-Task Reinforcement Learning. In *Proc. NIPS*, 2014.
- [59] E. Brunskill and L. Li. Sample Complexity of Multi-task Reinforcement Learning. In *UAI*, 2013.
- [60] M. Snel and S. Whiteson. Learning potential functions and their representations for multi-task reinforcement learning. *Auton Agent Multi-Agent Syst*, 2014.
- [61] M. E. Taylor and P. Stone. Transfer Learning for Reinforcement Learning Domains: A Survey. *JMLR*, 2009.
- [62] A. Lazaric. Transfer in Reinforcement Learning: a Framework and a Survey. *Reinforcement Learning - State of the art*, 2013.
- [63] Y. Zhan and M. E. Taylor. Online Transfer Learning in Reinforcement Learning Domains. In *AAAI*, 2015.
- [64] G. Konidaris, I. Scheidwasser, and A. G. Barto. Transfer in Reinforcement Learning via Shared Features. *JMLR*, 2012.
- [65] F. L. da Silva and A. H. R. Costa. Transfer Learning for Multiagent Reinforcement Learning Systems. In *Proc. IJCAI*, 2016.
- [66] X. Guo, S. Singh, H. Lee, R. Lewis, and X. Wang. Deep Learning for Real-Time Atari Game Play Using Offline Monte-Carlo Tree Search Planning. In *Proc. NIPS*, 2014.
- [67] G. Dulac-Arnold, H. van Hasselt R. Evans, P. Sunehag, J. Hunt T. Lillicrap, T. Mann, T. Weber, T. Degris, and B. Coppin. Deep Reinforcement Learning in Large Discrete Action Spaces. <https://arxiv.org/pdf/1512.07679v2.pdf>, 2015.
- [68] M. Hausknecht and P. Stone. Deep Reinforcement Learning in Parameterized Action Space. In *Proc. ICLR*, 2016.
- [69] M. Hausknecht and P. Stone. Deep Recurrent Q-Learning for Partially Observable MDPs. In <http://arxiv.org/abs/1507.06527>, 2015.
- [70] R. Guo and D. Hoiem. Support surface prediction in indoor scenes. In *Proc. ICCV*, 2013.
- [71] A. G. Schwing and R. Urtasun. Efficient Exact Inference for 3D Indoor Scene Understanding. In *Proc. ECCV*, 2012.
- [72] A. G. Schwing, T. Hazan, M. Pollefeys, and R. Urtasun. Efficient Structured Prediction for 3D Indoor Scene Understanding. In *Proc. CVPR*, 2012.
- [73] A. G. Schwing and R. Urtasun. Fully Connected Deep Structured Networks. In <https://arxiv.org/abs/1503.02351>, 2015.
- [74] U. Syed and R. E. Schapire. A Game-Theoretic Approach to Apprenticeship Learning. In *Proc. NIPS*, 2007.
- [75] M. Valko, M. Ghavamzadeh, and A. Lazaric. Semi-Supervised Apprenticeship Learning. In *JMLR*, 2012.

- [76] M. Babeş-Vroman, V. Marivate, K. Subramanian, and M. Littman. Apprenticeship Learning About Multiple Intentions. In *Proc. ICML*, 2011.
- [77] C. Daniel, M. Viering, J. Metz, O. Kroemer, and Jan Peters. Active Reward Learning. In *Proc. RSS*, 2010.
- [78] S. Levine, C. Finn, T. Darrell, and P. Abbeel. End-to-End Training of Deep Visuomotor Policies. In <https://arxiv.org/abs/1504.00702>, 2015.
- [79] R. Houthoofd, X. Chen, Y. Duan, J. Schulman, F. De Turck, and P. Abbeel. Variational Information Maximizing Exploration. In <https://arxiv.org/abs/1605.09674>, 2016.
- [80] Y. Duan, X. Chen, R. Houthoofd, J. Schulman, and P. Abbeel. Benchmarking Deep Reinforcement Learning for Continuous Control. In <https://arxiv.org/abs/1604.06778>, 2016.
- [81] I. Lenz, R. Knepper, and A. Saxena. DeepMPC: Learning Deep Latent Features for Model Predictive Control. In *Proc. RSS*, 2015.
- [82] V. Kumar, E. Todorov, and S. Levine. Optimal control with learned local models: Application to dexterous manipulation. In *Conf. on Robot. and Auto.*, 2016.
- [83] F. Zhang, J. Leitner, M. Milford, B. Upcroft, and P. Corke. Towards Vision-Based Deep Reinforcement Learning for Robotic Motion Control. In <https://arxiv.org/abs/1511.03791>, 2015.
- [84] N. Heess, G. Wayne, D. Silver, T. Lillicrap, Y. Tassa, and T. Erez. Learning Continuous Control Policies by Stochastic Value Gradients. In *Proc. NIPS*, 2015.
- [85] A. Y. Ng, D. Harada, and S. Russell. Policy Invariance Under Reward Transformations: Theory and Applications to Reward Shaping. In *Proc. ICML*, 1999.
- [86] E. Wiewiora, G. W. Cottrell, and C. Elkan. Principled methods for advising reinforcement learning agents. In *2003, Proc. ICML*.
- [87] E. Wiewiora. Potential-Based Shaping and Q-Value Initialization are Equivalent. In *JAIR*, 2003.
- [88] M. Grzes and D. Kudenko. Learning Shaping Rewards in Model-based Reinforcement Learning. In *Proc. AAMAS*, 2009.
- [89] M. Grzes and D. Kudenko. Online learning of shaping rewards in reinforcement learning. In *Proc. ICANN*, 2010.
- [90] S. Devlin, L. Yliniemi, D. Kudenko, and K. Tumer. Potential-Based Difference Rewards for Multiagent Reinforcement Learning. In *Proc. AAMAS*, 2014.
- [91] M. G. Bellemare, S. Srinivasan, G. Ostrovski, T. Schaul, D. Saxton, and R. Munos. Unifying Count-Based Exploration and Intrinsic Motivation. In <https://arxiv.org/abs/1606.01868>, 2016.
- [92] M. Lopes, F. Melo, and L. Montesano. Active Learning for Reward Estimation in Inverse Reinforcement Learning. In *Proc. ECML*, 2009.
- [93] S. Griffith, K. Subramanian, J. Scholz, C. Isbell, and A. Thomaz. Policy Shaping: Integrating Human Feedback with Reinforcement Learning. In *Proc. NIPS*, 2013.
- [94] W. B. Knox and P. Stone. Combining manual feedback with subsequent mdp reward signals for reinforcement learning. In *Proc. AAMAS*, 2010.

- [95] W. B. Knox and P. Stone. Interactively shaping agents via human reinforcement: The tamer framework. In *Conf. on Knowledge Capture*, 2009.
- [96] B. D. Argall, S. Chernova, M. Veloso, and B. Browning. A survey of robot learning from demonstration. In *Proc. Robot. Auton. Syst.*, 2009.
- [97] H. B. Suay, T. Brys, T. E. Taylor, and S. Chernova. Reward Shaping by Demonstration. 2015.
- [98] H. B. Suay, T. Brys, M. E. Taylor, and S. Chernova. Learning from Demonstration for Shaping through Inverse Reinforcement Learning. In *Proc. AAMAS*, 2016.
- [99] T. Brys, A. Harutyunyan, H. B. Suay, S. Chernova, M. E. Taylor, and A. Nowé. Reinforcement learning from demonstration through shaping. In *Proc. IJCAI*, 2015.
- [100] S. Devlin and D. Kudenko. Dynamic potential-based reward shaping. In *Proc. AAMAS*, 2012.
- [101] A. Harutyunyan, T. Brys, P. Vrancx, and A. Nowé. Shaping mario with human advice. In *Proc. AAMAS*, 2015.
- [102] A. Harutyunyan, S. Devlin, P. Vrancx, and A. Nowé. Expressing arbitrary reward functions as potential-based advice. In *Proc. AAAI*, 2015.
- [103] S. Levine and V. Koltun. Continuous inverse optimal control with locally optimal examples. In *Proc. ICML*, 2012.
- [104] K. Muelling, A. Boularias, B. Mohler, B. Schölkopf, and J. Peters. Learning strategies in table tennis using inverse reinforcement learning. In *Biological cybernetics*, 2014.
- [105] S. Schaal. Learning from demonstration. In *Proc. NIPS*, 1997.
- [106] M. E. Taylor and P. Stone. Cross-domain transfer for reinforcement learning. In *Proc. ICML*, 2007.